

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное
Образовательное учреждение высшего образования
МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
(МОСКОВСКИЙ ПОЛИТЕХ)

ДОПУЩЕН К ЗАЩИТЕ

Заведующий кафедрой

_____ / Пухова Е. А. /

Руководитель образовательной программы

_____ / Даньшина М. В. /

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
по теме:
**РАЗРАБОТКА КЛИЕНТ-СЕРВЕРНОЙ СИСТЕМЫ
КОРПОРАТИВНОГО ОБУЧЕНИЯ С ПОДДЕРЖКОЙ АДАПТИВНОГО
ТЕСТИРОВАНИЯ**

по направлению 09.03.03 Прикладная информатика
Образовательная программа (профиль) «Корпоративные информационные
системы»

Студент: _____ / Мурадян Лусинэ Гамлетовна, 221–361/
подпись *ФИО, группа*

Руководитель ВКР: _____ / Гонтовой Сергей Викторович, доцент, к.н./
подпись *ФИО, уч. звание и степень*

Москва 2026

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное
Образовательное учреждение высшего образования
МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
(МОСКОВСКИЙ ПОЛИТЕХ)

ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ
по направлению 09.03.03 Прикладная информатика
Образовательная программа (профиль) «Корпоративные информационные
системы»

Тема ВКР	Разработка клиент-серверной системы корпоративного обучения с поддержкой адаптивного тестирования
ПРАКТИЧЕСКИЙ РЕЗУЛЬТАТ	
Назначение	Клиент-серверная система корпоративного обучения с поддержкой адаптивного тестирования для персонализации учебного процесса сотрудников и студентов, а также предоставления организациям удобных средств управления курсами и аналитики результатов обучения.
Основные функции	1. Управление пользователями и ролями (обучающийся, преподаватель, администратор организации, системный администратор). 2. Создание и редактирование курсов, уроков и тестов преподавателями. 3. Назначение курсов пользователям и группам администраторами. 4. Прохождение курсов и тестов обучающимися с мобильного приложения. 5. Адаптивное тестирование с подстройкой сложности заданий по результатам. 6. Формирование рекомендаций по повторению материалов. 7. Аналитика и отчёты по прогрессу для администраторов и преподавателей. 8. Поддержка нескольких организаций (multi tenant архитектура).
Используемые технологии и платформы	Сервер: Python (Django REST Framework / FastAPI), PostgreSQL Мобильное приложение: Flutter (Dart) Веб-панель: React (JavaScript/TypeScript)

	Протоколы: REST-API, HTTPS Развёртывание: облачная инфраструктура
ВЫПОЛНЕНИЕ РАБОТЫ	
Решаемые задачи	1. Анализ предметной области корпоративного электронного обучения и существующих LMS. 2. Анализ подходов к адаптивному тестированию и персонализации обучения. 3. Проектирование функциональности системы и ролей пользователей. 4. Проектирование архитектуры клиент серверной системы и базы данных. 5. Разработка модуля адаптивного тестирования и рекомендаций. 6. Проектирование и реализация мобильного приложения обучающегося. 7. Разработка веб панели администратора и преподавателя. 8. Реализация серверной части с REST API. 9. Тестирование системы и анализ информационной безопасности.
Состав технической документации	1. Техническое задание на разработку системы. 2. Программа и методика испытаний. 3. Руководство пользователя (мобильное приложение). 4. Руководство администратора (веб-панель). 5. Описание серверной части и API. 6. Спецификация требований к системе.
Состав графической части	1. Список графической части

ПЛАН РАБОТЫ НАД ВКР

[illegible]

РУКОВОДИТЕЛЬ ОП:

«___»_____2026, _____ / Даньшина Марина Владимировна /
подпись *ФИО, уч. звание и степень*

РУКОВОДИТЕЛЬ ВКР:

«___»_____2026, _____ / Гонтовой Сергей Викторович, доцент, к.н. /
подпись *ФИО, уч. звание и степень*

СТУДЕНТ:

«___»_____2026, _____ / Мурадян Лусинэ Гамлетовна, 221–361/ /
подпись *ФИО, группа*

АННОТАЦИЯ

Наименование работы: выпускная квалификационная работа на тему «Разработка клиент-серверной системы корпоративного обучения с поддержкой адаптивного тестирования».

Цель работы — разработать клиент-серверную систему корпоративного обучения с поддержкой адаптивного тестирования для персонализации учебного процесса сотрудников и студентов. Способы достижения: анализ предметной области и аналогов, формирование требований и сценариев использования, проектирование архитектуры и модели данных, описание алгоритма адаптивного тестирования, подготовка рекомендаций по внедрению и безопасности.[1, 2, 3, 4]

Практический результат — прототип LMS с мультиарендной (multi-tenant) архитектурой, адаптивным тестированием и удобным интерфейсом для мобильного и веб-доступа. Новизна и ценность состоят в комплексной проработке мобильного UX, встроенной адаптивной модели подбора заданий и проектировании единого серверного решения для нескольких организаций.[5]

Работа состоит из введения и двух глав (предметная область и технологии; проектирование системы), списка использованных источников и приложений. Во введении обоснованы актуальность, цель и задачи работы, объект и предмет исследования. В первой главе описаны предметная область, постановка задачи, анализ аналогов и выбор технологий. Во второй главе представлены сценарии использования, архитектура и модель данных (диаграммы компонентов, процессов и БД), описание двух алгоритмов (адаптивный подбор вопросов и изоляция данных по тенантам), а также макеты основных страниц (прототипы интерфейсов).

Библиографический список состоит из источников по мере их упоминания в тексте (печатные и электронные в соответствии с ГОСТ 7.05-2008).

Результаты работы могут быть апробированы в рамках внедрения прототипа в учебный процесс или представлены в виде доклада на конференции.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	9
1 ПРЕДМЕТНАЯ ОБЛАСТЬ И ТЕХНОЛОГИИ	12
1.1 Предметная область корпоративного обучения и исходное состояние	12
1.2 Постановка проблемы и назначение системы	14
1.3 Требования к системе	17
1.4 Анализ аналогов	24
1.5 Выбор технологий и платформ	26
1.6 Выводы по главе	31
2 ТЕХНИЧЕСКАЯ РЕАЛИЗАЦИЯ СИСТЕМЫ	32
2.1 Архитектура системы	32
2.2 Сценарии использования	34
2.3 Модель данных	37
2.4 Проектирование интерфейсов	40
2.5 Реализация web- и mobile-клиентов	43
2.6 Реализация серверной логики и алгоритмов	47
2.7 Развёртывание и использование	52
2.8 Выводы по главе	53
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	54

ВВЕДЕНИЕ

Выпускная квалификационная работа посвящена разработке корпоративной системы управления обучением, предназначенной для организации учебного контента, назначения курсов, контроля результатов и проведения адаптивного тестирования в рамках единой клиент–серверной платформы.[5]

Актуальность темы обусловлена тем, что в корпоративной среде обучение сотрудников всё чаще строится как непрерывный процесс, включающий онбординг, обязательные программы, проверку знаний и повторение проблемных тем. При использовании разрозненных инструментов усложняется управление пользователями и ролями, затрудняется назначение курсов, отсутствует единый контур аналитики и слабо поддерживается персонализация учебной траектории. Дополнительным фактором является распространение мобильного обучения, при котором слушатели проходят курсы короткими сессиями со смартфона и ожидают простой навигации, быстрого доступа к материалам и прозрачной обратной связи по результатам тестирования.[2, 6]

Для корпоративных сценариев также существенна возможность обслуживания нескольких организаций в рамках одной платформы с логическим разделением данных, ролей и учебного контента. Такой подход позволяет использовать единое программное решение для нескольких арендаторов, сохраняя изоляцию данных и независимость администрирования.[7, 4]

Целевой аудиторией разрабатываемой системы являются системный администратор платформы, администратор организации, преподаватель и слушатель. Для административных ролей важны средства настройки организаций, пользователей, курсов и аналитики, а для слушателя — удобное прохождение назначенного обучения с мобильного устройства, получение результата тестирования и рекомендаций по повторению материала.

Цель работы — разработать корпоративную LMS с веб-панелью администрирования и мобильным клиентом слушателя, обеспечивающую управ-

ление учебным контентом, назначениями, адаптивным тестированием и аналитикой результатов обучения.

Для достижения поставленной цели необходимо решить следующие задачи:

- а) проанализировать предметную область корпоративного обучения и исходное состояние процессов;
- б) выявить основные проблемы организации учебного процесса и обосновать необходимость автоматизации;
- в) сформулировать функциональные и нефункциональные требования к системе;
- г) выполнить сравнительный анализ существующих LMS-решений и определить критерии сопоставления;
- д) обосновать выбор технологий и платформ для разработки программного комплекса;
- е) спроектировать архитектуру системы, модель данных и пользовательские сценарии;
- ж) реализовать серверную, веб- и мобильную части системы;
- з) описать алгоритмы адаптивного тестирования, формирования рекомендаций и изоляции данных арендаторов;
- и) сформировать сценарий тестирования требований по критическому пути пользователей и сценарий демонстрации текущей версии проекта.

Объектом исследования являются процессы организации корпоративного обучения, контроля прохождения курсов и оценки результатов слушателей в цифровой образовательной среде.

Предметом исследования являются методы и средства проектирования и разработки клиент–серверной информационной системы корпоративного обучения, включающей веб-интерфейс администрирования, мобильный клиент слушателя, адаптивное тестирование и механизмы аналитики.

Практическим результатом работы является программный комплекс, реализованный в виде монорепоzitория и включающий серверное прило-

жение на основе FastAPI, веб-панель администратора и преподавателя на React и TypeScript, а также мобильное приложение слушателя на Flutter. Серверная часть обеспечивает аутентификацию, управление организациями, пользователями, курсами, уроками, тестами, рекомендациями и аналитикой. В системе реализованы разграничение доступа по ролям, логическая multi-tenant изоляция данных, адаптивный подбор вопросов в тестах и формирование рекомендаций по слабым темам.

Практическая значимость полученного результата заключается в том, что он демонстрирует возможность организации корпоративного обучения в едином цифровом контуре: от подготовки контента и назначения курсов до прохождения адаптивного тестирования и анализа результатов. Для предметной области это означает сокращение ручных операций, повышение прозрачности контроля обучения и возможность дальнейшего развития системы под задачи конкретной организации.

Структура работы соответствует поставленным задачам. Во введении обоснованы актуальность темы, определены цель, задачи, объект, предмет исследования и описан практический результат. В первой главе рассматриваются предметная область корпоративного обучения, постановка проблемы, требования к системе, аналоги и обоснование выбора технологий. Во второй главе описываются архитектура системы, пользовательские сценарии, модель данных, проектирование интерфейсов, серверная логика, алгоритмы, а также особенности развёртывания и использования программного комплекса. Список использованных источников оформлен в соответствии с требованиями методических рекомендаций.[5]

1 ПРЕДМЕТНАЯ ОБЛАСТЬ И ТЕХНОЛОГИИ

1.1 Предметная область корпоративного обучения и исходное состояние

Системы управления обучением применяются для централизованного хранения учебных материалов, назначения курсов, контроля прохождения и фиксации результатов. В корпоративной среде LMS используются для он-бординга, обязательного обучения, внутренних регламентов, курсов по информационной безопасности и повышения квалификации сотрудников.[1, 8]

Для рассматриваемой предметной области характерно наличие нескольких устойчивых ролей. Системный администратор поддерживает общесистемные настройки и контролирует работу платформы. Администратор организации управляет пользователями, ролями, курсами и назначениями в пределах своей организации. Преподаватель создаёт учебный контент, тесты и анализирует результаты. Слушатель проходит назначенные курсы, изучает уроки, выполняет тесты и получает рекомендации по слабым темам.

В исходном состоянии процесс обучения часто строится с использованием нескольких несвязанных инструментов: документы и видео хранятся отдельно, списки обучающихся ведутся вручную, результаты тестирования не агрегируются, а повторение слабых тем организуется без формальной аналитики. Это приводит к ряду практических проблем:

- а) отсутствует единый источник данных о пользователях, курсах и результатах;
- б) повышаются трудозатраты на назначение курсов и контроль прохождения;
- в) преподаватель не получает целостной картины по ошибкам и слабым темам;
- г) слушатель проходит одинаковые по сложности тесты вне зависимости от уровня подготовки;

д) использование мобильных устройств поддерживается ограниченно или неудобно.

На рисунке 1 представлена схема процесса корпоративного обучения в цифровой среде. Диаграмма показывает последовательность действий от подготовки контента и назначения курса до прохождения тестирования и анализа результатов.

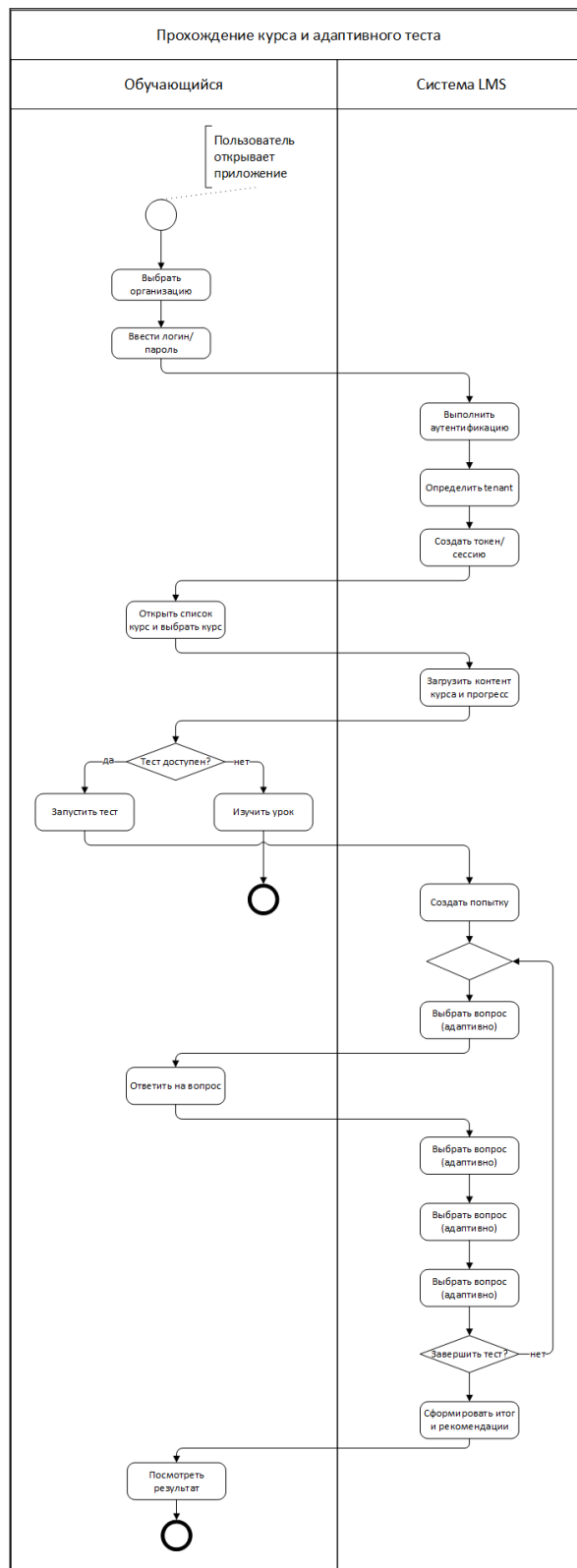


Рисунок 1 – Схема процесса корпоративного обучения в цифровой среде

1.2 Постановка проблемы и назначение системы

Проблема разработки состоит в необходимости объединить в одной системе функции администрирования, хранения контента, назначения учеб-

ных программ, мобильного доступа, тестирования и анализа результатов без усложнения пользовательских сценариев. Типовые LMS нередко обладают избыточным или трудно настраиваемым функционалом, а их мобильный пользовательский опыт не всегда соответствует сценарию коротких учебных сессий.[9, 6]

Назначение разрабатываемой системы заключается в поддержке полного цикла корпоративного обучения:

- а) создание и редактирование пользователей, ролей, курсов, уроков и тестов;
- б) назначение курсов слушателям;
- в) прохождение уроков и фиксирование прогресса;
- г) запуск адаптивного теста и подбор вопросов с учётом текущей сложности;
- д) формирование рекомендаций по слабым темам;
- е) просмотр аналитики по обучению в административном интерфейсе.

Система должна обеспечивать работу нескольких организаций в одном развёртывании. В проекте эта задача решается не путём создания отдельной базы данных на каждого арендатора, а через логическую multi-tenant модель: идентификация организации в запросе, проверка участия пользователя в организации и фильтрация данных по полю `tenant_id`. Такой подход достаточен для учебного MVP и соответствует текущей реализации проекта.

На рисунке 2 показана диаграмма вариантов использования, отражающая ключевые действия ролей в системе. Диаграмма фиксирует распределение функций между системным администратором, администратором организации, преподавателем и слушателем.

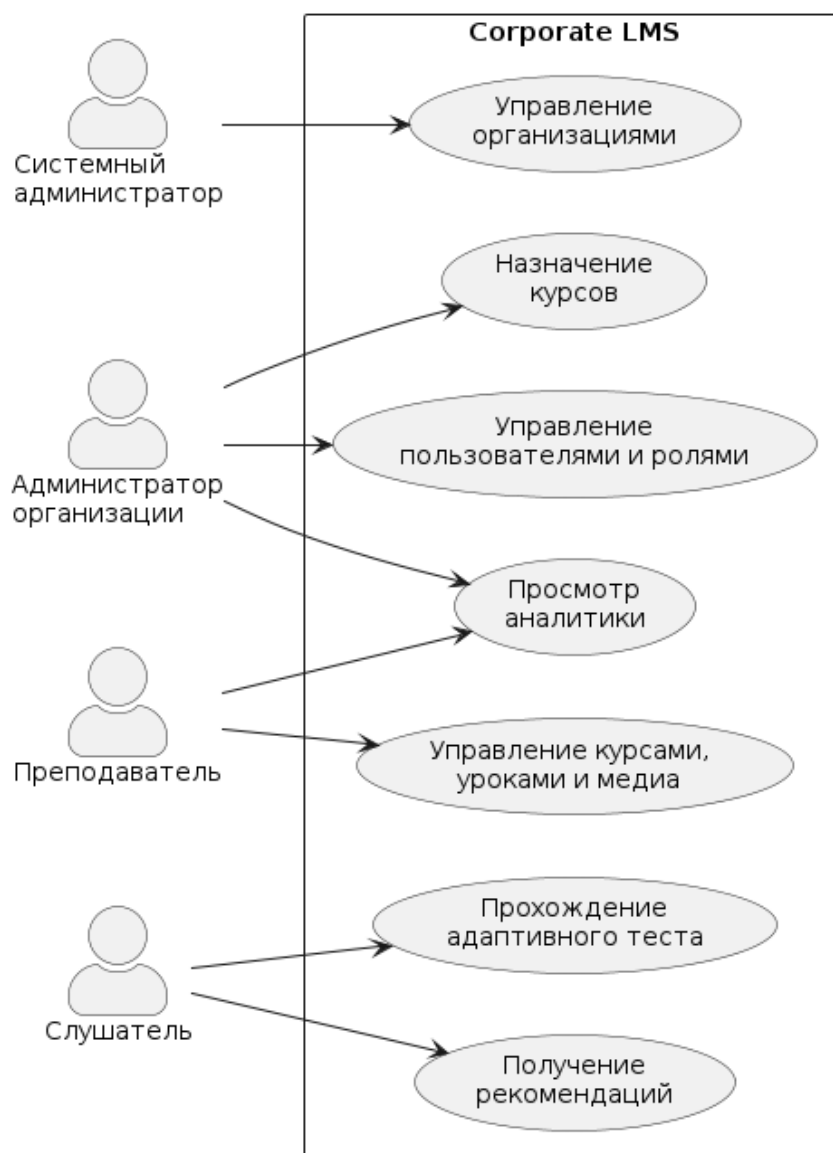


Рисунок 2 – Диаграмма вариантов использования системы

Для более формального представления сценариев в таблице 1 приведены основные прецеденты системы, их акторы и ожидаемые результаты выполнения.

Таблица 1 – Ключевые прецеденты системы

Прецедент	Основной актер	Результат выполнения
Управление организациями	Системный администратор	Созданы и настроены организации, доступные для дальнейшей работы пользователей.
Управление пользователями и ролями	Администратор организации	В организации созданы пользователи и назначены роли в соответствии с полномочиями.
Управление курсами, уроками и медиа	Преподаватель	Подготовлен учебный контент, доступный для назначения слушателям.
Назначение курса	Администратор организации	Выбранный курс назначен слушателю и появился в его списке обучения.
Прохождение адаптивного теста	Слушатель	Сформирована завершённая попытка с результатом и историей ответов.
Получение рекомендаций	Слушатель	Пользователь получает перечень тем и материалов для повторения.
Просмотр аналитики	Преподаватель, администратор организации	Получены агрегированные показатели по курсам, попыткам и успеваемости.

1.3 Требования к системе

Требования к программному продукту сформированы с учётом методических рекомендаций по ВКР и общепринятых принципов тестирования требований.[5, 10] В рамках анализа требований важно не только перечислить ожидаемую функциональность, но и проверить требования на завершённость, непротиворечивость, атомарность, недвусмысленность и проверяемость.[10]

1.3.1 Уровни и типы требований

Для разрабатываемой корпоративной LMS требования целесообразно рассматривать на нескольких уровнях: бизнес-требования, пользовательские требования и требования к продукту.[10] Такой подход позволяет связать цель проекта с ролями пользователей и с конкретной реализуемой функциональностью.

Бизнес-требование. Организация должна получить единый программный комплекс, обеспечивающий управление корпоративным обучением, назначение курсов, контроль прохождения, адаптивную проверку знаний и анализ результатов без использования набора разрозненных инструментов.

Пользовательские требования. На пользовательском уровне требования фиксируют задачи, которые выполняют роли системы, и поэтому в работе представлены в виде вариантов использования, табличных прецедентов и формализованных сценариев.

а) системный администратор должен иметь возможность управлять организациями и общесистемными настройками;

б) администратор организации должен иметь возможность управлять пользователями, ролями и назначениями курсов в пределах своей организации;

в) преподаватель должен иметь возможность создавать курсы, уроки, тесты и анализировать результаты обучения;

г) слушатель должен иметь возможность проходить назначенные курсы, выполнять тестирование и получать рекомендации по слабым темам.

Требования к продукту. На уровне продукта требования разделяются на функциональные и нефункциональные. Дополнительно в проекте выделяются требования к интерфейсам, требования к данным и ограничения, так как именно эти группы напрямую влияют на архитектуру системы, структуру API и модель хранения данных.

1.3.2 Функциональные требования

Функциональные требования сгруппированы по подсистемам.

Подсистема аутентификации и выбора организации

а) Система должна обеспечивать регистрацию и аутентификацию пользователей.

б) Система должна поддерживать выдачу токенов доступа и обновления.

в) Система должна определять текущую организацию по контексту запроса и проверять, что пользователь имеет активное участие в выбранной организации.

Подсистема управления пользователями и ролями

- а) Администратор организации должен иметь возможность создавать пользователей.
- б) Администратор организации должен иметь возможность изменять состояние доступа пользователя.
- в) Система должна поддерживать роли слушателя, преподавателя, администратора организации и системного администратора.

Подсистема управления курсами и уроками

- а) Преподаватель и администратор должны иметь возможность создавать и редактировать курсы.
- б) Преподаватель и администратор должны иметь возможность создавать и редактировать уроки.
- в) Система должна поддерживать хранение структурированного контента урока, изображений и видео.

Подсистема тестирования

- а) Преподаватель и администратор должны иметь возможность создавать тесты и вопросы.
- б) Система должна поддерживать указание базовой сложности теста и лимита вопросов.
- в) Система должна обеспечивать запуск попытки, выдачу следующего вопроса, приём ответа и завершение попытки.

Подсистема адаптивного тестирования и рекомендаций

- а) После каждого ответа система должна изменять текущую сложность попытки на один уровень вверх или вниз.
- б) Система должна выбирать следующий вопрос среди ещё не заданных вопросов, наиболее близких к текущей сложности.
- в) Система должна выявлять слабые темы по ошибочным и медленным ответам.
- г) После завершения теста система должна формировать рекомендации по повторению материала.

Подсистема аналитики

- а) Система должна предоставлять агрегированные показатели по пользователям, курсам, тестам и назначенным обучением.
- б) Система должна отображать проблемные темы и индивидуальные результаты слушателя.
- в) Система должна обеспечивать просмотр истории попыток и детального разбора попытки.

1.3.3 Нефункциональные требования

Безопасность

- а) Пароли пользователей должны храниться в виде криптографических хэшей.
- б) Доступ к защищённым операциям должен выполняться только после аутентификации и проверки роли.
- в) Система должна вести аудит ключевых действий пользователей.

Изоляция данных

- а) Данные разных организаций не должны пересекаться при выполнении запросов.
- б) Любая tenant-зависимая сущность должна быть связана с идентификатором организации.

Производительность и сопровождаемость

- а) Время отклика на типовые операции чтения должно быть приемлемым для интерактивной работы пользователя.
- б) Серверная часть должна быть структурирована по подсистемам и слоям, упрощающим сопровождение и развитие.
- в) Система должна поддерживать контейнеризированное локальное развёртывание.

Удобство использования и кроссплатформенность

а) Веб-интерфейс должен быть ориентирован на административные сценарии.

б) Мобильный интерфейс должен быть ориентирован на сценарии слушателя.

в) Платформа должна поддерживать как минимум две клиентские платформы: web и mobile.

1.3.4 Требования к интерфейсам, данным и ограничения

В дополнение к функциональным и нефункциональным требованиям целесообразно зафиксировать связанные группы требований, уточняющие границы решения и способ взаимодействия подсистем.

Требования к интерфейсам

а) Взаимодействие web- и mobile-клиентов с серверной частью должно выполняться через REST API с передачей данных в формате JSON.

б) Защищённые операции должны выполняться только после аутентификации пользователя по токену доступа.

в) Для tenant-зависимых операций система должна определять текущую организацию по контексту запроса и не допускать обращения к данным другой организации.

Требования к данным

а) Tenant-зависимые сущности должны быть связаны с идентификатором организации `tenant_id`.

б) Система должна хранить структуру курсов и уроков, вопросы тестов, историю попыток, результаты проверки, слабые темы и сформированные рекомендации.

в) Ролевая модель должна храниться отдельно от учетной записи пользователя и учитывать участие пользователя в одной или нескольких организациях.

Ограничения

а) Система разрабатывается как клиент–серверный программный комплекс, включающий backend, web-клиент и mobile-клиент.

б) В текущей реализации изоляция данных арендаторов должна обеспечиваться логически в рамках общего экземпляра базы данных, а не путём выделения отдельной БД на каждого арендатора.

в) Локальное развёртывание должно быть воспроизводимым и выполняться средствами контейнеризации.

1.3.5 Свойства качественных требований

С точки зрения методики анализа требований требования к системе должны удовлетворять базовым качественным свойствам.[10]

Таблица 2 – Проверка набора требований на свойства качества

Свойство	Применение к требованиям системы
Завершённость	Для каждой подсистемы указаны действия пользователя и ожидаемый результат системы.
Атомарность	Требования сформулированы как отдельные проверяемые утверждения без объединения разных сценариев в одном пункте.
Непротиворечивость	Роли и права доступа согласованы с фактической моделью RBAC, реализованной в проекте.
Недвусмысленность	Избегаются выражения вида “удобный”, “быстрый”, “интеллектуальный” без контекста проверки.
Прослеживаемость	Требования связаны с ролями, прецедентами, критическим путём пользователей и архитектурными решениями, что позволяет проследить путь от цели системы до проверки реализации.
Модифицируемость	Требования сгруппированы по подсистемам и типам, что упрощает их расширение, корректировку и локализацию изменений.
Выполнимость	Формулировки ограничены фактическими возможностями текущего проекта, используемым стеком и принятой логической multi-tenant моделью.
Обязательность и актуальность	В перечень включены только требования, непосредственно связанные с целью ВКР, ролями системы и критическим пользовательским сценарием.
Корректность	Требования описывают поведение системы и её свойства, а не действия пользователя, и не содержат заявлений о функциях, отсутствующих в реализации.
Проверяемость	Для критических функций возможно построить тестовые сценарии и получить однозначный результат проверки.

1.3.6 Сценарий тестирования требований по критическому пути пользователей

В качестве критического пути пользователей выбран сквозной сценарий, затрагивающий ключевые роли и основные бизнес-функции системы. Такой сценарий позволяет связать требования, архитектурные решения и проверку реализации в единую последовательность действий.

- а) преподаватель или администратор организации входит в систему;
- б) преподаватель создаёт курс, уроки и тест;
- в) администратор назначает курс слушателю;
- г) слушатель проходит урок и запускает адаптивный тест;
- д) система фиксирует попытку, изменяет сложность вопросов, завершает тест и формирует рекомендации;
- е) администратор или преподаватель просматривает аналитику и результаты слушателя.

Данный сценарий позволяет проверить полноту и согласованность требований, поскольку в нём задействованы создание контента, назначение обучения, прохождение курса, серверная логика тестирования, рекомендации и аналитика. В таблице 3 приведена матрица трассировки критического пути, связывающая шаги сценария с группами требований и ожидаемыми результатами проверки.

Таблица 3 – Матрица трассировки критического пути пользователей

Шаг	Роль	Проверяемые требования	Ожидаемый результат
1	Преподаватель или администратор организации	Аутентификация, выбор организации, разграничение прав доступа	Пользователь получает доступ к функциям своей роли в пределах выбранной организации.
2	Преподаватель	Управление курсами, уроками, тестами и медиа	Учебный контент создан и подготовлен к назначению слушателю.
3	Администратор организации	Назначение курса, tenant-изоляция и контроль ролей	Выбранный курс становится доступным назначенному слушателю.
4	Слушатель	Просмотр уроков, запуск теста, выполнение попытки	Слушатель проходит урок и запускает адаптивное тестирование.
5	Система	Изменение сложности, фиксация попытки, выявление слабых тем, формирование рекомендаций	Сохраняются результат, история ответов, слабые темы и рекомендации по повторению материала.
6	Преподаватель или администратор организации	Аналитика, история попыток, детализированный просмотр результатов	Административная роль получает агрегированные и детализированные данные по обучению.

На рисунке 3 показана иерархия ключевых требований системы, используемая как схема вертикальной трассировки от цели проекта к основным группам функций.

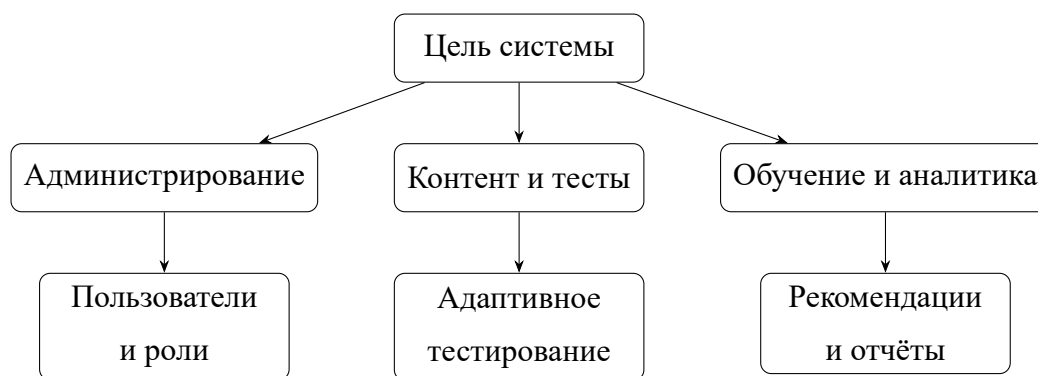


Рисунок 3 – Иерархия ключевых требований системы

1.4 Анализ аналогов

Для сравнения выбраны решения, представляющие разные классы LMS: открытые платформы (*Moodle*, *Open LMS*), облачный вариант на базе Moodle (*MoodleCloud*), платформы массовых онлайн-курсов и коммерческие корпоративные LMS.[1, 8, 11, 12]

Сравнение выполняется по критериям, значимым для данной ВКР:

- а) наличие административных ролей и разделения прав доступа;
- б) удобство мобильного доступа для слушателя;
- в) наличие или отсутствие адаптивного тестирования;
- г) возможности аналитики и работы со слабыми темами;
- д) поддержка multi-tenant сценария;
- е) пригодность для учебной разработки и доработки архитектуры.

Поскольку административные роли поддерживаются всеми рассматриваемыми решениями, в таблице 4 сопоставление выполнено по критериям, по которым платформы различаются сильнее всего.

Таблица 4 – Сравнение аналогов по ключевым критериям

Решение	Мобильный доступ	Адаптивное тестирование	Аналитика	Multi-tenant
Moodle	средне	ограничено	да	ограничено
Open LMS	средне	ограничено	да	ограничено
MoodleCloud	средне	ограничено	да	ограничено
Коммерческие LMS	хорошо	частично или да	хорошо	да
Собственная система	хорошо	да	да	да

По результатам сравнения можно сделать следующие выводы. Moodle и производные от него платформы обеспечивают широкий базовый функционал, однако требуют более сложной настройки и не ориентированы на демонстрацию собственной архитектуры в рамках учебной разработки. Коммерческие LMS предлагают зрелые средства мобильного доступа и аналитики, но являются закрытыми и хуже подходят как основа выпускного проекта. Разрабатываемая система уступает крупным платформам по степени зрелости, но лучше соответствует целям ВКР, поскольку позволяет явно продемонстрировать архитектуру, модель данных, адаптивное тестирование и tenant-изоляцию.

Результаты сравнения показывают, что существующие решения закрывают базовые задачи LMS, однако либо обладают избыточностью и сложностью настройки, либо не ориентированы на исследование и реализацию собственной архитектуры. Для целей ВКР целесообразно разрабатывать соб-

ственный программный комплекс, в котором можно явно продемонстрировать клиент–серверную архитектуру, модель данных, алгоритм адаптивного тестирования и изоляцию данных арендаторов.

1.5 Выбор технологий и платформ

Выбор технологий определялся не только их распространённостью, но и соответствием требованиям проекта. Разрабатываемая система должна поддерживать отдельные web- и mobile-клиенты, REST API, tenant-изоляцию данных, развитие модели предметной области и воспроизводимое локальное развёртывание. Поэтому для каждой подсистемы рассматривались несколько вариантов, после чего выбиралось решение, лучше всего соответствующее целям ВКР и текущему масштабу проекта.

1.5.1 Критерии выбора технологий

При выборе технологического стека учитывались следующие критерии:

- а) соответствие клиент–серверной архитектуре с отдельными web- и mobile-клиентами;
- б) удобство реализации REST API, аутентификации, разграничения доступа и tenant-контекста;
- в) возможность формального описания модели данных и сопровождения изменений схемы БД;
- г) пригодность для MVP, который необходимо не только разработать, но и продемонстрировать на защите;
- д) воспроизводимость локального развёртывания без сложной инфраструктуры.

1.5.2 Сравнение технологий и результат выбора

Сравнение основных вариантов приведено в таблице 5. В таблицу включены именно те подсистемы, которые определяют архитектурный облик проекта и влияют на удобство дальнейшей доработки.

Таблица 5 – Сравнение технологий и результаты выбора

Подсистема	Рассмотренные варианты	Выбранная технология и причина
Серверная часть	FastAPI, Django REST Framework, Flask	Выбран FastAPI, так как проект имеет формат API-first и не требует серверной генерации HTML. По сравнению с Django REST Framework решение получается легче и проще для MVP, а по сравнению с Flask требует меньше ручной сборки маршрутов, схем и документации.
Доступ к данным	SQLAlchemy + Alembic, Django ORM, прямой SQL	Выбраны SQLAlchemy и Alembic, поскольку они не привязаны к конкретному веб-фреймворку, позволяют явно описывать модели и контролировать миграции. Прямой SQL усложнил бы сопровождение, а Django ORM логичнее использовать только вместе с Django.
Web-клиент	React + TypeScript, Vue, Angular	Выбраны React и TypeScript. Для административной панели важны компонентный подход, маршрутизация, работа с формами и типизация данных. Angular для MVP избыточен, а Vue также подходит, но в текущем проекте более удобным оказался гибкий компонентный подход React.
Мобильный клиент	Flutter, React Native, native Android	Выбран Flutter, поскольку он позволяет поддерживать единый код мобильного приложения и быстро собрать интерфейс слушателя. Нативная разработка увеличила бы трудоёмкость, а React Native не давал принципиального преимущества при уже разделённых web- и mobile-сценариях.
Хранение данных	SQLite, PostgreSQL, MySQL	Используется комбинированный подход: SQLite удобен для локальной разработки и тестирования без отдельного сервера, а PostgreSQL лучше подходит для контейнеризованного запуска и демонстрации многопользовательской системы.
Развёртывание	ручной запуск сервисов, Docker Compose, Kubernetes	Выбран Docker Compose, так как он достаточен для MVP и позволяет поднимать backend, web и базу данных одной конфигурацией. Ручной запуск хуже воспроизводим, а Kubernetes для учебного проекта избыточен.

1.5.3 Обоснование выбора серверной части

Для серверной части рассматривались три основных подхода: использование полноценного фреймворка Django с Django REST Framework, минималистичного Flask и API-ориентированного FastAPI. В рамках данной ВКР сервер выполняет роль централизованного REST API для двух клиентов и не требует шаблонной серверной генерации страниц. По этой причине возможности Django, связанные с административной панелью, шаблонами и встроенной ORM, для проекта оказались избыточными. Flask, напротив, предоставляет слишком низкий уровень абстракции, из-за чего значительную часть инфраструктурного кода пришлось бы собирать вручную.[13, 14]

FastAPI оказался наиболее сбалансированным вариантом. Он хорошо подходит для явного описания маршрутов, схем запросов и ответов, зависимостей и проверки входных данных. Это соответствует фактической реализации backend, где сервер разделён на группы маршрутов auth, tenants, users, courses, media, tests, recommendations и analytics. Такая структура хорошо сочетается с модульным монолитом и позволяет расширять систему без переработки базовой архитектуры.[13]

Для доступа к данным выбраны SQLAlchemy и Alembic. Такой набор позволяет описывать сущности предметной области на уровне моделей, а изменения схемы фиксировать через миграции. Для ВКР это важно, поскольку система включает достаточно большую группу взаимосвязанных сущностей: организации, пользователей, роли, курсы, уроки, тесты, попытки, рекомендации и аналитические данные. Использование прямого SQL на всём протяжении проекта усложнило бы сопровождение и повторяемость изменений, тогда как связка SQLAlchemy + Alembic обеспечивает более управляемое развитие схемы.

1.5.4 Обоснование выбора технологий клиентских приложений

В административной части проекта рассматривались React + TypeScript, Vue и Angular. Для web-панели требуются страницы авторизации, управления пользователями, курсами, уроками, тестами, назначениями и аналитикой. Такой интерфейс состоит из большого числа форм, таблиц и маршрутов, которые должны переиспользовать общие компоненты и работать с типизированными данными. По этой причине выбор был сделан в пользу React как гибкой компонентной библиотеки, а TypeScript был добавлен для уменьшения количества ошибок при работе с API-моделями, ролями, курсами и результатами тестирования.[15, 16]

Angular предоставляет более жёсткую архитектурную модель, однако для MVP это увеличивает объём шаблонного кода и порог входа. Vue также является сильным кандидатом, но в рассматриваемом проекте приоритет был отдан экосистеме React, которая хорошо сочетается с компонентной структурой административной панели, маршрутизацией и быстрым циклом сборки через Vite. Таким образом, выбор React + TypeScript оказался наиболее удобным именно для текущего состава интерфейсных сценариев.

Для мобильного клиента рассматривались Flutter, React Native и нативная Android-разработка. Так как в рамках ВКР необходимо было показать самостоятельный mobile-клиент слушателя, поддерживающий авторизацию, просмотр курсов, воспроизведение медиа, прохождение тестов и получение рекомендаций, требовалось решение с единым кодом приложения и предсказуемой сборкой интерфейса. Flutter отвечает этим требованиям и позволяет реализовать мобильный интерфейс как самостоятельную часть комплекса, не смешивая его с web-клиентом.[17]

Нативная Android-разработка на Kotlin обеспечила бы высокий уровень платформенной интеграции, но заметно увеличила бы трудоёмкость проекта. React Native позволил бы использовать близкий по идеологии стек, однако не снимал бы задачи отдельной мобильной адаптации и не давал

бы существенного выигрыша по сравнению с уже выбранным web-стеком. Поэтому для данной ВКР Flutter оказался более рациональным выбором.

1.5.5 Выбор СУБД и средств развёртывания

На уровне хранения данных в проекте используется двухконтурный подход. Для локальной разработки и части тестовых сценариев применяется SQLite, так как она не требует запуска отдельного сервера, упрощает старт проекта и ускоряет проверку изменений. Для контейнеризированного развёртывания используется PostgreSQL, так как серверная СУБД лучше отражает рабочий режим многопользовательской системы, где одновременно функционируют backend, web и база данных.[18, 19]

Такое сочетание не является противоречием. Напротив, оно позволяет использовать простой режим разработки на ранних этапах и более реалистичный режим демонстрации на этапе развёртывания. Наличие в проекте конфигураций и для SQLite, и для PostgreSQL подтверждает, что выбранный стек ориентирован как на удобство разработки, так и на последующую демонстрацию результата.

Для локального развёртывания рассматривались ручной запуск сервисов, Docker Compose и более тяжёлые оркестрационные решения. Ручной запуск backend, web и базы данных неудобен для повторяемой демонстрации, так как требует последовательной настройки каждого компонента. Использование Kubernetes для учебного MVP не даёт соразмерной выгоды и существенно усложняет инфраструктурную часть работы. По этой причине в проекте выбран Docker Compose, который позволяет в одном файле конфигурации описать сервисы базы данных, backend, web и при необходимости сервис начального заполнения данными.[20]

Таким образом, текущий стек выбран не случайно, а как результат сопоставления требований проекта и трудоёмкости реализации. Он соответствует фактической структуре репозитория, обеспечивает поддержку web- и mobile-

платформ, удобен для развития предметной модели и позволяет воспроизводить демонстрировать программный комплекс в рамках ВКР.

1.6 Выводы по главе

В первой главе рассмотрена предметная область корпоративного обучения, выявлены проблемы исходного состояния процессов и сформулировано назначение разрабатываемой системы. Определены функциональные и нефункциональные требования, приведена их проверка на основные свойства качества, а также сформирован сценарий тестирования требований по критическому пути пользователей. Выполнено сравнение аналогов и обоснован выбор собственного программного решения. На основании требований и анализа существующих платформ определён технологический стек проекта, соответствующий фактической реализации и задачам выпускной квалификационной работы.

2 ТЕХНИЧЕСКАЯ РЕАЛИЗАЦИЯ СИСТЕМЫ

2.1 Архитектура системы

Разрабатываемая система реализована как модульный монолит с REST API, объединённый в одном репозитории, но логически разделённый на серверную часть, веб-клиент и мобильный клиент. Такой подход упрощает сопровождение учебного проекта, позволяет явно показать взаимодействие подсистем и при этом сохраняет понятную декомпозицию по функциональным модулям.

Серверная часть выступает единственным источником бизнес-логики и данных. Она отвечает за аутентификацию, выбор текущей организации, проверку ролей, работу с курсами и уроками, управление тестами, выполнение адаптивного тестирования, формирование рекомендаций и подготовку аналитики. Веб-клиент ориентирован на роли администратора и преподавателя. Мобильный клиент ориентирован на роль слушателя и закрывает сценарии прохождения обучения.

На рисунке 4 представлена контейнерная схема системы. Диаграмма показывает состав программного комплекса на уровне контейнеров и основные каналы взаимодействия между web-клиентом, mobile-клиентом, backend API, базой данных и хранилищем медиафайлов.

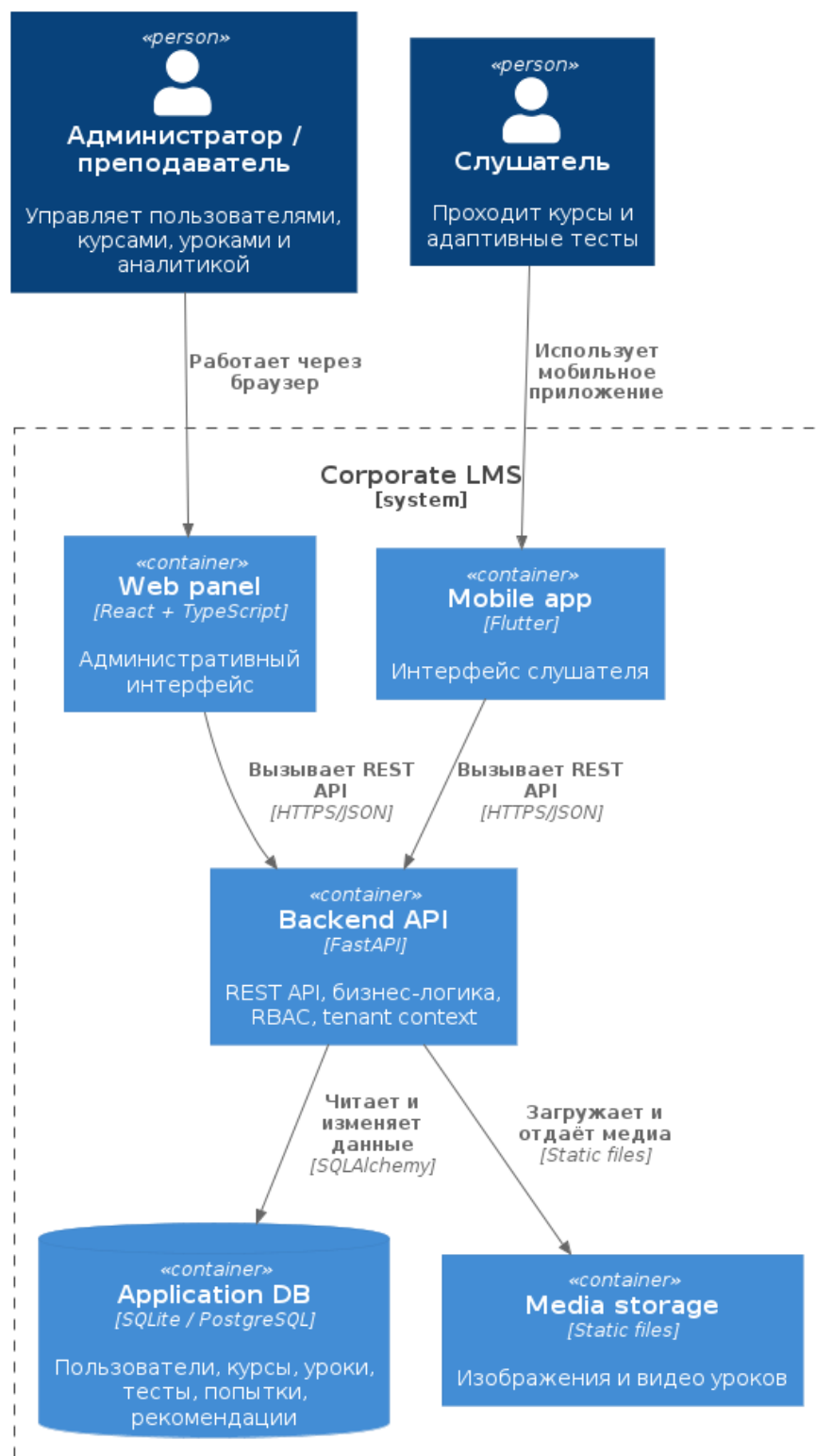


Рисунок 4 – Контейнерная схема программного комплекса

С точки зрения программной структуры backend включает следующие подсистемы:

- а) auth — аутентификация, токены доступа и профиль пользователя;
- б) tenants — работа с организациями и текущим tenant-контекстом;
- в) users — управление пользователями и их состоянием;

- г) `courses` и `lessons` — управление курсами, уроками и прогрессом;
- д) `media` — загрузка и выдача медиафайлов;
- е) `tests` — жизненный цикл тестов и попыток;
- ж) `adaptive` — выбор следующего вопроса и расчёт рекомендаций;
- з) `recommendations` — выдача списка рекомендаций слушателю;
- и) `analytics` — агрегированная аналитика по системе.

На рисунке 5 приведена диаграмма компонентов backend-части. Она отражает декомпозицию серверного приложения на функциональные подсистемы и их связи с общими инфраструктурными механизмами.

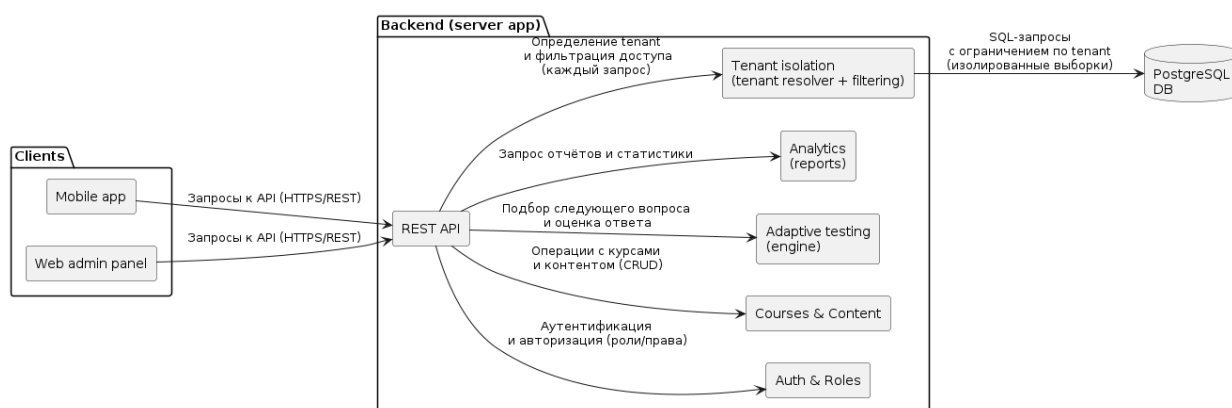


Рисунок 5 — Диаграмма компонентов системы

2.2 Сценарии использования

Во второй главе сценарии использования описываются как формальные последовательности действий, служащие основой для архитектурных и интерфейсных решений. Такое представление позволяет зафиксировать роли, предусловия, основной поток и результат выполнения для каждого ключевого сценария.

На основании диаграммы прецедентов и критического пути пользователей в таблице 6 приведено табличное представление ключевых сценариев, используемое для согласования ролей, предусловий и результатов выполнения.

Таблица 6 – Табличное представление основных сценариев использования

Сценарий	Актор	Предусловие	Результат
Подготовка обучения и назначение курса	Преподаватель, администратор организации	Пользователь аутентифицирован и работает в контексте своей организации	Курс создан, наполнен контентом и назначен слушателю.
Прохождение курса и адаптивного теста	Слушатель	Слушатель включён в организацию и имеет назначенный курс	Сформирован результат попытки и набор рекомендаций по слабым темам.
Анализ результатов обучения	Преподаватель, администратор организации	В системе накоплены данные по назначениям и попыткам	Получены агрегированные показатели и детализированные результаты по слушателям.

2.2.1 Сценарий 1: подготовка обучения и назначение курса

Акторы: администратор организации, преподаватель.

Предусловия: пользователь аутентифицирован, организация выбрана, роль пользователя позволяет управлять контентом и назначениями.

Основной поток:

- а) преподаватель создаёт курс;
- б) преподаватель добавляет уроки и при необходимости медиа-контент;
- в) преподаватель создаёт тест и вопросы;
- г) администратор организации выбирает слушателя;
- д) администратор назначает курс слушателю;
- е) система фиксирует назначение и делает курс доступным в мобильном клиенте слушателя.

Постусловия: слушатель получает назначенный курс и может приступить к изучению материала.

2.2.2 Сценарий 2: прохождение курса и адаптивного теста

Актор: слушатель.

Предусловия: слушатель аутентифицирован, имеет активное участие в организации, курс назначен, тест связан с курсом.

Основной поток:

- а) слушатель открывает список своих курсов;
- б) слушатель переходит в курс и изучает урок;
- в) слушатель запускает тест;
- г) система создаёт попытку с базовой сложностью;
- д) система выдаёт первый вопрос;
- е) слушатель отправляет ответ;
- ж) система оценивает ответ и изменяет текущую сложность;
- з) система выдаёт следующий вопрос с учётом новой сложности;
- и) после достижения лимита вопросов система завершает попытку;
- к) система сохраняет результат, слабые темы и формирует рекомендации.

Постусловия: результат попытки, история сложности и рекомендации доступны слушателю и административным ролям.

2.2.3 Сценарий 3: анализ результатов обучения

Актёры: преподаватель, администратор организации.

Основной поток:

- а) пользователь открывает раздел аналитики;
- б) система отображает агрегированные показатели по курсам, тестам и назначенным обучением;
- в) пользователь открывает проблемные темы или индивидуальный отчёт по слушателю;
- г) система отображает результаты попыток, количество правильных ответов и рекомендации.

2.3 Модель данных

Модель данных построена вокруг сущностей, необходимых для поддержки multi-tenant обучения. На концептуальном уровне она отражает объекты предметной области и их семантические связи, а на логическом уровне фиксирует состав реляционных таблиц, ключей и ограничений целостности. Центральной сущностью является организация (tenant), относительно которой изолируются пользователи, курсы, уроки, тесты, попытки и рекомендации.

2.3.1 Инфологическая модель базы данных

Инфологическая модель описывает систему без привязки к конкретной СУБД и показывает укрупнённые сущности предметной области: организацию, пользователя, курс, урок, тему, тест, вопрос, попытку и результат. Такая степень абстракции позволяет наглядно показать логику движения данных от подготовки учебного контента до прохождения адаптивного теста и получения результата.

На рисунке 6 приведена инфологическая модель базы данных. На этом уровне намеренно опущены служебные и ассоциативные сущности, связанные с ролями, назначениями, рекомендациями и аудитом, так как они раскрываются в даталогической модели.

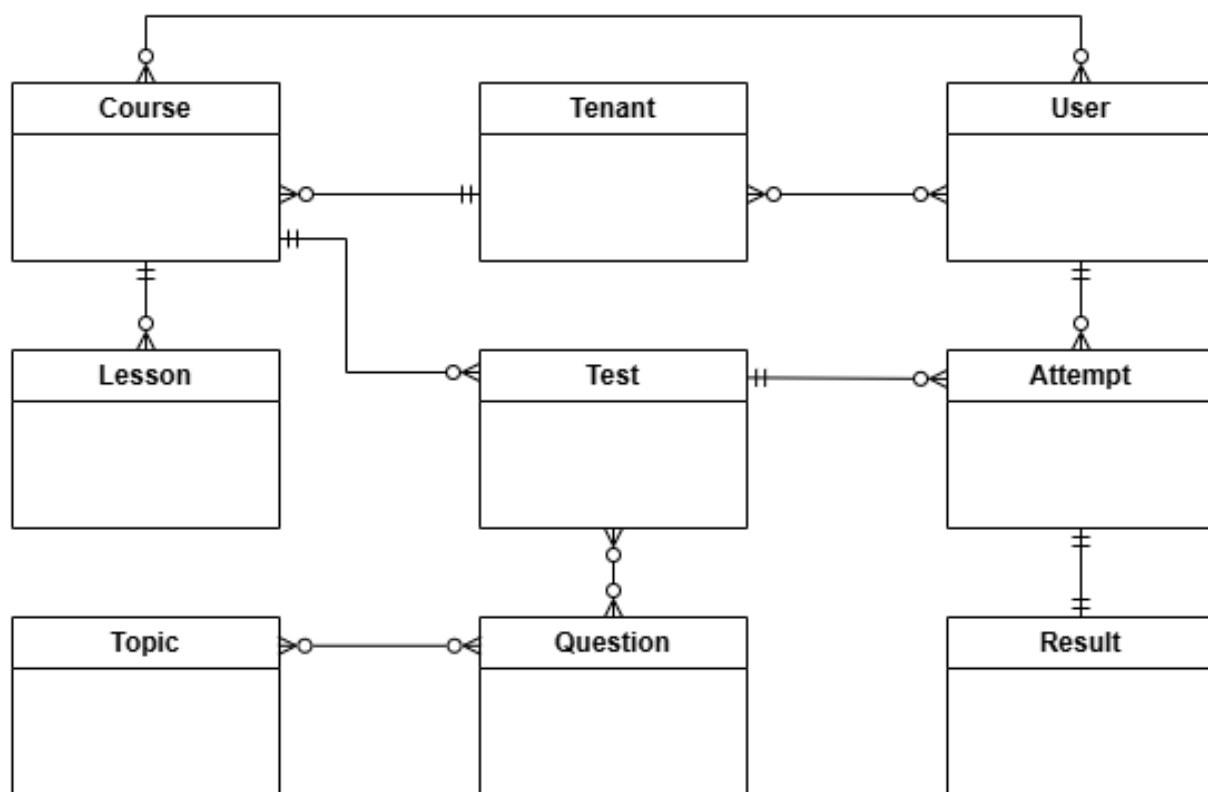


Рисунок 6 – Инфологическая модель базы данных

2.3.2 Даталогическая модель базы данных

На даталогическом уровне инфологическая модель преобразуется в реляционную схему, реализованную средствами SQLAlchemy. При локальном запуске приложение может использовать SQLite, а при контейнеризированном развёртывании — PostgreSQL; при этом набор таблиц, первичных и внешних ключей остаётся одинаковым. Для tenant-зависимых сущностей в схему включено поле `tenant_id`, а для критичных связей заданы ограничения уникальности, исключающие дублирование членств, назначений и записей о прогрессе.

Для пояснительной записки даталогическая модель представлена двумя основными фрагментами, отражающими базовые контуры проектируемой системы: организационный контур и контур управления учебным контентом. Такой способ позволяет показать структуру таблиц и связей без перегрузки рисунков второстепенными служебными сущностями.

На рисунке 7 показана даталогическая модель организационного контура. Она описывает, каким образом в системе представлены организации, пользователи, роли, членства и учебные группы. Данный фрагмент показывает реализацию multi-tenant подхода: один пользователь может состоять в нескольких организациях, а его права определяются через таблицу членства и связанную с ней роль. Дополнительно здесь отражено объединение пользователей в группы, используемые при массовом назначении обучения.

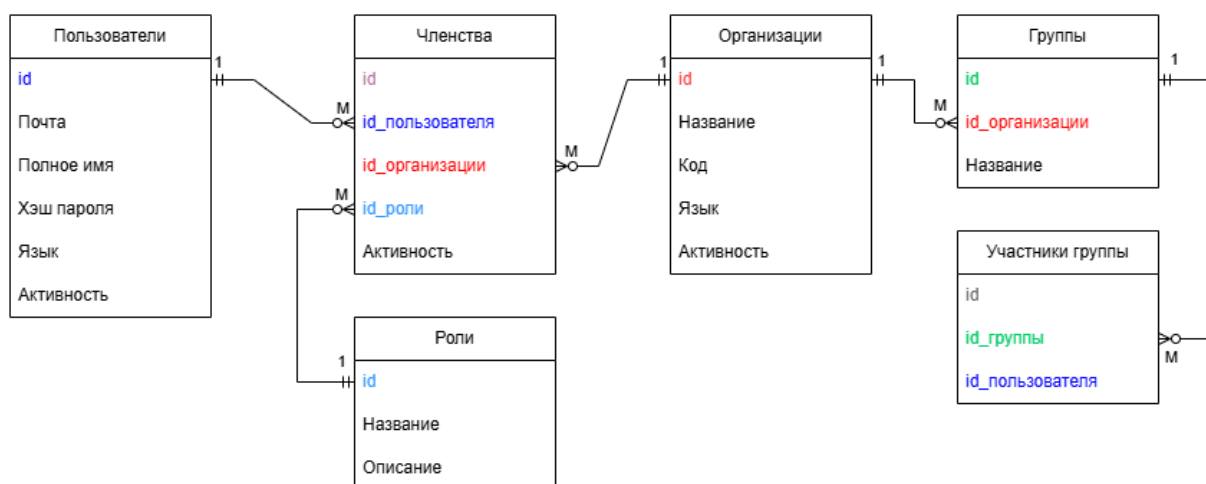


Рисунок 7 – Даталогическая модель организационного контура

На рисунке 8 представлена даталогическая модель контура курсов. В неё включены таблицы, описывающие учебный контент, структуру курса, назначения, зачисления и фиксацию прогресса. Схема показывает, что курс принадлежит организации, состоит из уроков и тем, может назначаться как отдельному пользователю, так и группе, а факт прохождения отражается в таблицах зачислений и прогресса по урокам. Такой контур обеспечивает хранение как структуры учебных материалов, так и состояния их освоения слушателями.

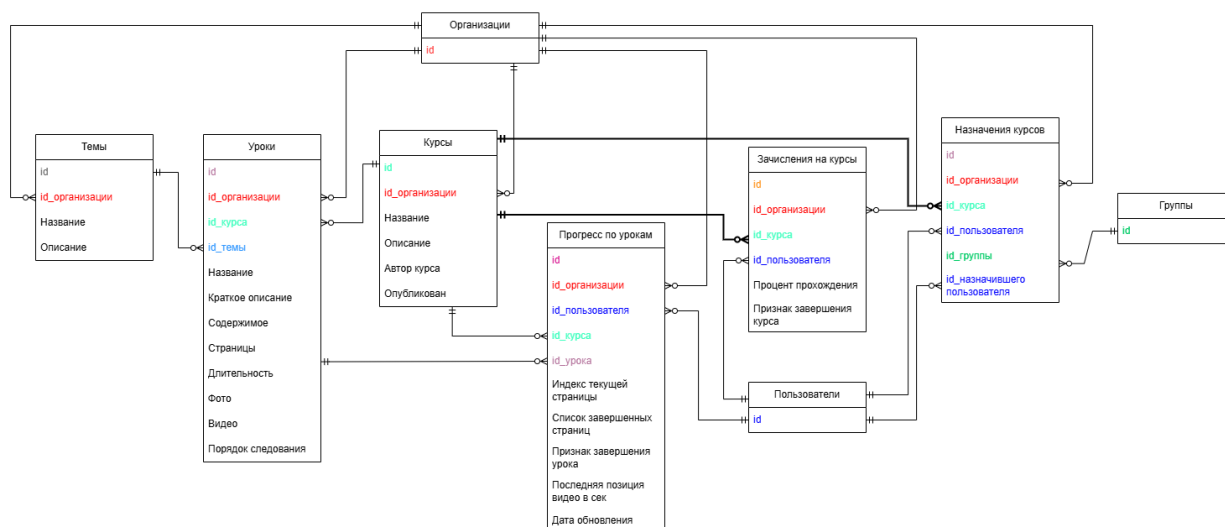


Рисунок 8 – Даталогическая модель контура курсов

Таким образом, в текущей версии пояснительной записки даталогическая модель раскрыта через два ключевых фрагмента: организационный и учебный. Первый описывает разграничение доступа и принадлежность пользователей к организациям и группам, а второй — структуру курсов, механизм их назначения и фиксацию прогресса обучения. Контуры тестирования, рекомендаций и служебных таблиц при необходимости могут быть представлены отдельно без перегрузки основной части работы.

2.4 Проектирование интерфейсов

Проектирование интерфейсов выполнялось отдельно для административного и пользовательского контуров.

2.4.1 Веб-интерфейс администратора и преподавателя

Веб-клиент предназначен для работы с учебным контентом и аналитикой. Его основными разделами являются:

- а) вход в систему и выбор организации;
- б) управление пользователями;
- в) управление курсами;
- г) управление уроками и медиатекой;

- д) управление тестами;
- е) назначение курсов;
- ж) просмотр аналитики;
- з) настройки профиля и языка интерфейса.

Структура веб-интерфейса ориентирована на последовательное выполнение административных операций: сначала создаётся контент, затем выполняются назначения, после чего доступны данные аналитики.

2.4.2 Мобильный интерфейс слушателя

Мобильный клиент ориентирован на компактный пользовательский сценарий:

- а) вход пользователя;
- б) просмотр списка назначенных курсов;
- в) открытие курса и урока;
- г) запуск теста;
- д) просмотр результата, рекомендаций и истории попыток.

На рисунках 9–10 показаны примеры экранов мобильного клиента.

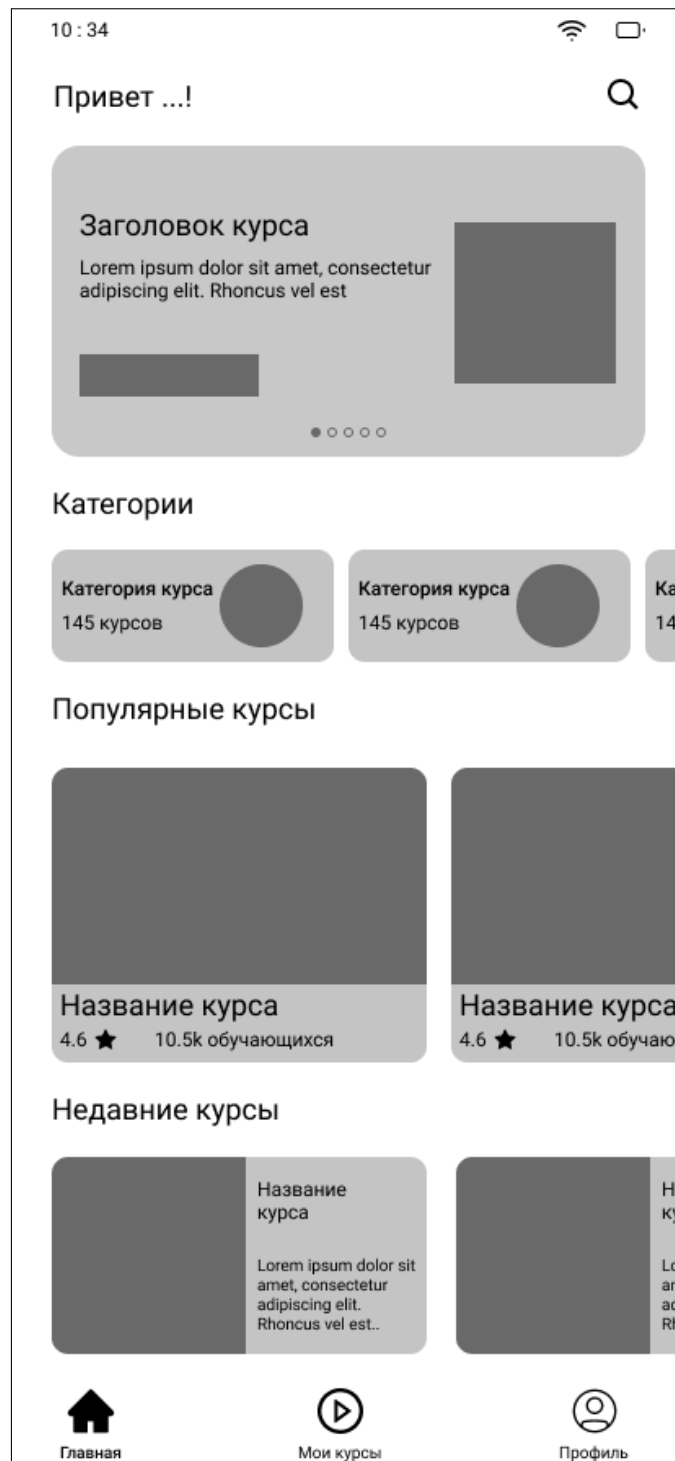


Рисунок 9 – Главный экран мобильного клиента

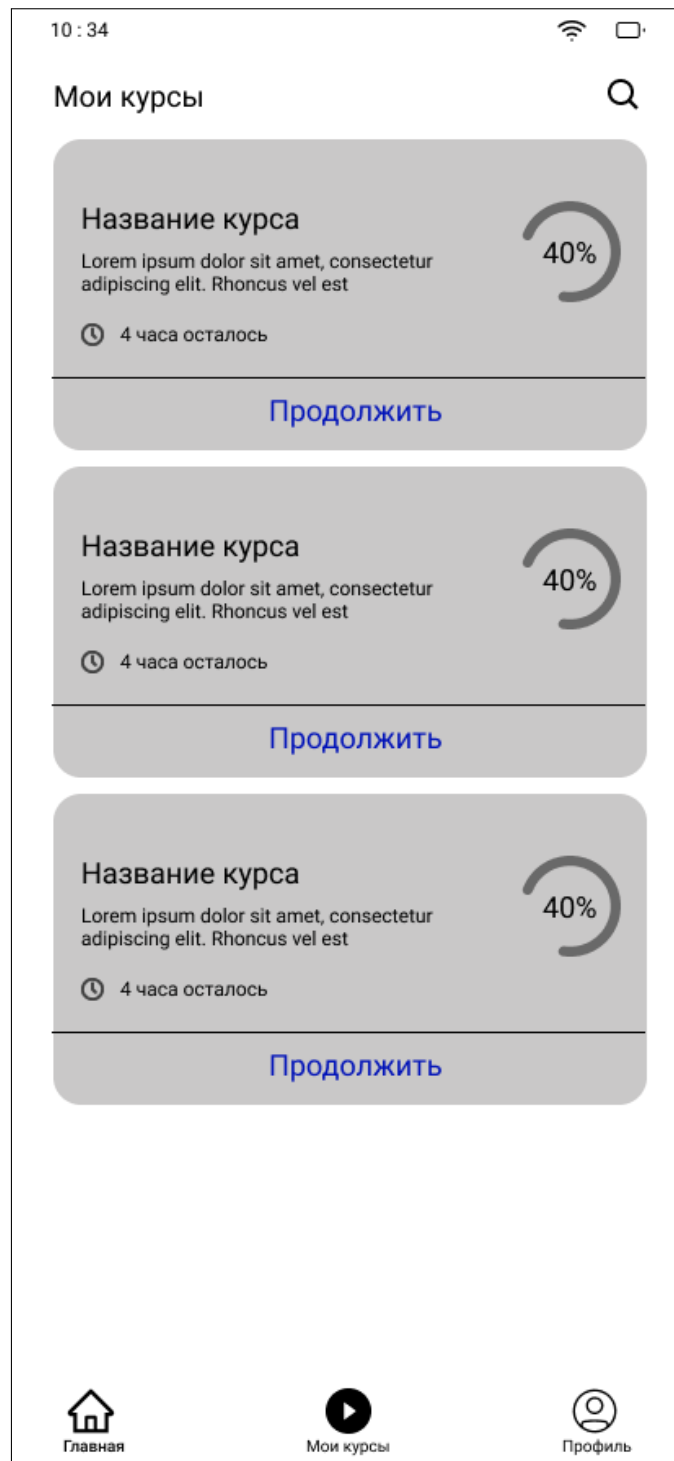


Рисунок 10 – Экран списка назначенных курсов

2.5 Реализация web- и mobile-клиентов

Клиентская часть системы реализована как два самостоятельных приложения, использующих единый backend API, но ориентированных на разные роли и сценарии работы. Веб-клиент предназначен для администратора и преподавателя, а мобильный клиент — для слушателя. Такое разделение

позволяет не перегружать интерфейсы лишними функциями и адаптировать взаимодействие с системой под реальные задачи пользователей.

На рисунке 11 представлена структурная схема клиентской части системы. Она показывает внутреннее устройство web- и mobile-приложений: пользовательский интерфейс, прикладные экраны, слой взаимодействия с backend API и локальное хранение пользовательского состояния. Общим для обоих клиентов является единый контракт обмена данными, а различие определяется набором экранов и способом управления состоянием.

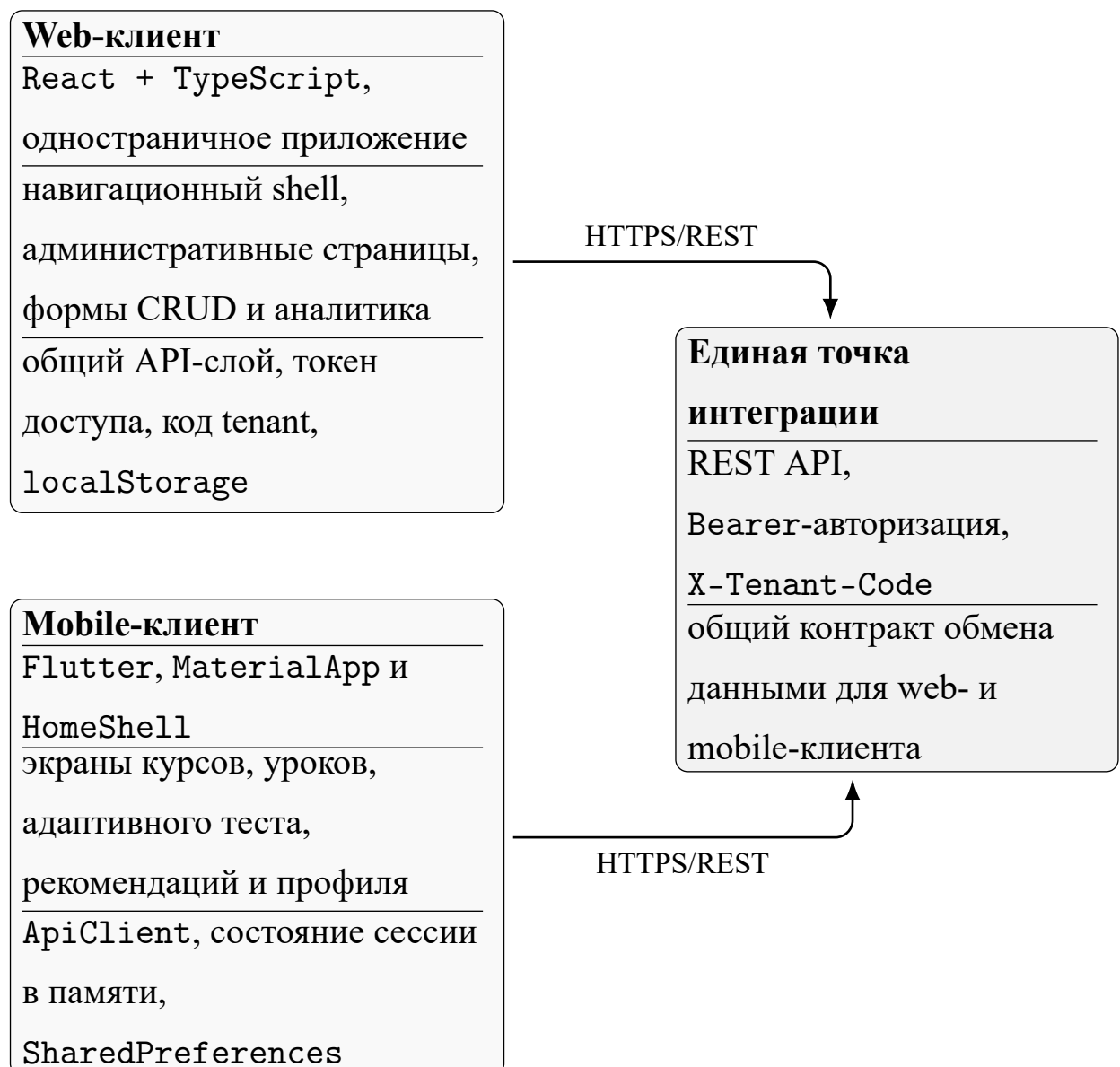


Рисунок 11 – Структурная схема клиентской части системы

2.5.1 Реализация web-клиента

Веб-клиент реализован как одностраничное приложение на React и TypeScript. После аутентификации пользователь попадает в общий каркас приложения с боковой навигацией, из которого доступны основные разделы: панель показателей, переключение организации, пользователи, курсы, уроки, тесты, назначения, аналитика и настройки. Маршрутизация организована на стороне клиента, что позволяет быстро переходить между разделами без повторной загрузки страницы.

Работа с backend вынесена в отдельный API-слой. Для всех запросов централизованно задаются базовый адрес сервера, заголовок авторизации Bearer и заголовок X-Tenant-Code, определяющий текущую организацию. Такой подход уменьшает дублирование кода и обеспечивает единое поведение для операций чтения, создания, редактирования и загрузки медиафайлов. Для типовых запросов используются отдельные функции отправки GET, POST, PATCH и multipart/form-data.

Сессионное состояние web-клиента включает токен доступа и код текущей организации. Эти данные передаются между страницами через состояние верхнего уровня приложения. Дополнительно в интерфейсе реализовано переключение языка, причём выбранный язык сохраняется в localStorage браузера и применяется при следующем открытии приложения.

Функциональные страницы web-клиента построены по единому принципу: загрузка данных через общий механизм запросов, отображение списка сущностей, форма создания или редактирования и визуальное подтверждение результата операции. Так реализованы управление пользователями, курсами, уроками, тестами и назначениями. В разделе уроков предусмотрено редактирование структуры страниц урока и загрузка медиафайлов, а в разделе аналитики — загрузка агрегированных показателей, проблемных тем и данных по конкретному слушателю. В результате веб-клиент закрывает весь ад-

министративный контур системы и напрямую поддерживает сценарии, описанные в первой главе.

2.5.2 Реализация мобильного клиента

Мобильный клиент реализован на Flutter как отдельное приложение слушателя. Корневой экран приложения построен на базе MaterialApp, а после входа пользователь переходит в основной HomeShell, содержащий нижнюю навигацию между тремя разделами: курсы, рекомендации и профиль. Для переключения между разделами используется IndexedStack, что позволяет сохранять состояние экранов при переходах.

Взаимодействие с сервером инкапсулировано в классе ApiClient. Он отвечает за формирование HTTP-запросов, добавление заголовков авторизации и tenant-контекста, обработку ошибок и преобразование JSON-ответов в модели приложения. Через этот клиент выполняются аутентификация, загрузка курсов, получение структуры курса, получение рекомендаций, работа с попытками тестирования и загрузка данных для разбора завершённых попыток.

Состояние мобильного приложения разделено на два уровня. Языковые настройки сохраняются в SharedPreferences, благодаря чему выбор русского или английского языка восстанавливается после перезапуска приложения. Сессионные данные текущего пользователя передаются между экранами в памяти приложения, что достаточно для учебного MVP и упрощает жизненный цикл клиента.

Экран списка курсов загружает назначенные пользователю курсы и открывает детальный экран курса. На экране курса выполняется параллельная загрузка структуры курса и связанных с ним тестов, после чего слушателю становятся доступны три действия: продолжение обучения, запуск адаптивного теста и просмотр истории попыток. Для прохождения урока используется отдельный экран-плеер, который получает данные урока с сервера и сохра-

няет прогресс пользователя, включая текущую страницу, набор завершённых страниц и позицию воспроизведения видео.

Отдельный экран `TestFlowScreen` реализует клиентскую часть адаптивного тестирования. Он инициирует попытку, получает очередной вопрос, отправляет выбранный ответ с учётом времени реакции и завершает попытку после исчерпания набора вопросов. После завершения теста слушатель видит результат, слабые темы и рекомендации. Дополнительно в мобильном клиенте реализованы экран рекомендаций и экран истории попыток с возможностью открыть детальный разбор завершённого теста. Таким образом, мобильный клиент не ограничивается просмотром контента, а поддерживает полный пользовательский цикл слушателя: от изучения курса до анализа собственных результатов.

2.6 Реализация серверной логики и алгоритмов

2.6.1 Реализация API-подсистем

Серверная часть реализует REST API, сгруппированное по назначению маршрутов. Такое разбиение определяет формат взаимодействия клиентов с backend и набор доступных операций.

- а) `/auth` — регистрация, вход, обновление токенов, получение профиля;
- б) `/tenants` — список доступных организаций, текущая организация, выбор организации;
- в) `/users` — просмотр, создание и изменение пользователей;
- г) `/courses` и `/lessons` — работа с курсами, уроками, прогрессом и назначениями;
- д) `/media` — библиотека и загрузка медиа;
- е) `/tests` и `/attempts` — тесты, вопросы, попытки, история и разбор;
- ж) `/recommendations` — рекомендации слушателя;
- з) `/analytics` — сводная и детальная аналитика.

Такое разбиение соответствует структуре кода backend и обеспечивает независимое развитие функциональных подсистем при сохранении единой точки входа.

На рисунке 12 приведена структурная схема серверной части системы. В отличие от общей диаграммы компонентов, эта схема акцентирует именно уровни обработки запроса: маршрутный слой, контур tenant-контекста и разграничения доступа, сервисный слой бизнес-логики и слой хранения данных.

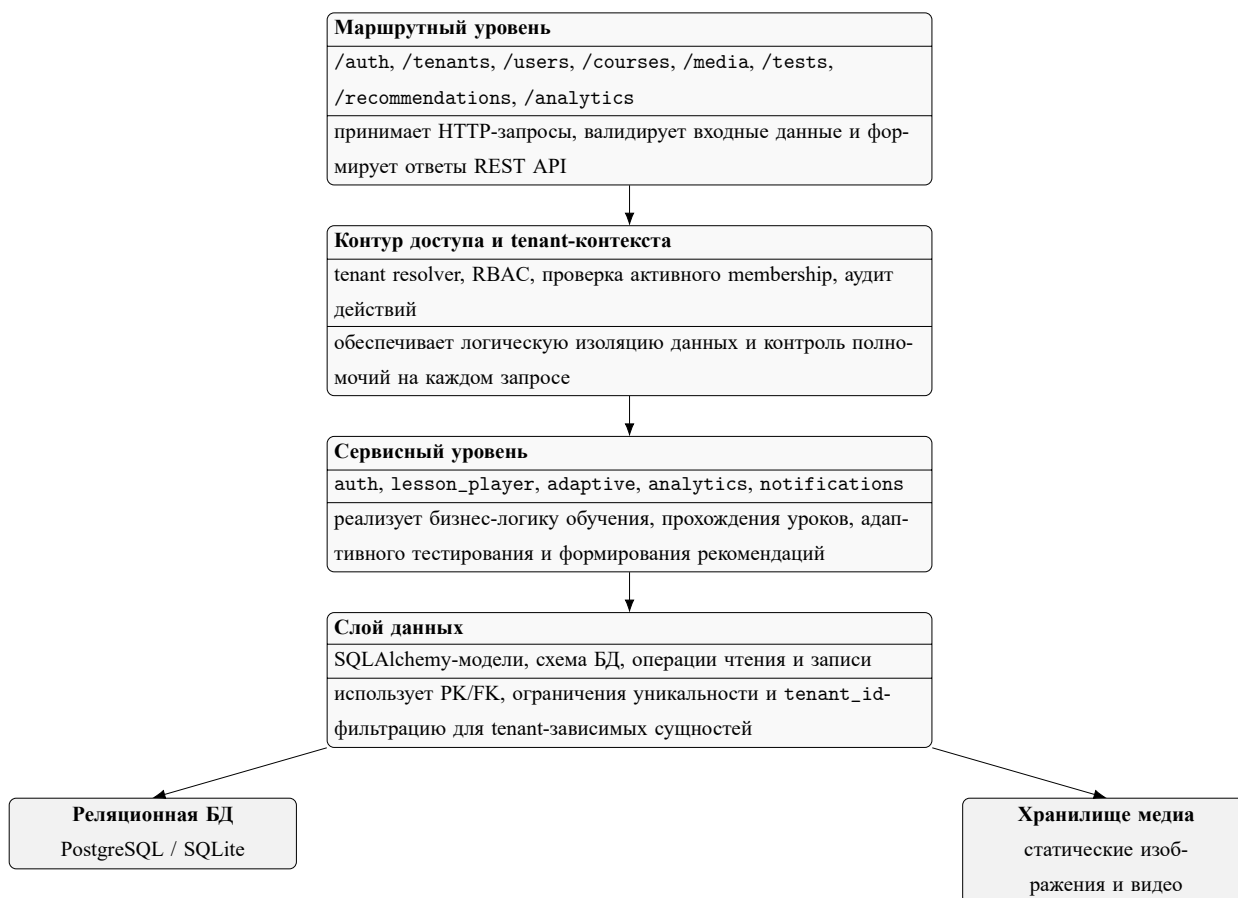


Рисунок 12 – Структурная схема серверной части системы

2.6.2 Алгоритм адаптивного тестирования

В проекте реализован дискретный алгоритм адаптивного тестирования, работающий в диапазоне сложностей от 1 до 5. Алгоритм основан на следующих правилах:

а) при старте попытки текущая сложность равна базовой сложности теста;

- б) после правильного ответа сложность повышается на один уровень;
- в) после неправильного ответа сложность понижается на один уровень;
- г) следующее задание выбирается среди ещё не заданных вопросов, наиболее близких к текущей сложности;
- д) по ошибочным и медленным ответам накапливаются слабые темы;
- е) после завершения попытки формируются рекомендации по повторению материала.

На рисунке 13 показана последовательность взаимодействия клиента и сервера при прохождении адаптивного теста. Диаграмма фиксирует обмен запросами между мобильным приложением, backend API и модулем адаптивной логики.

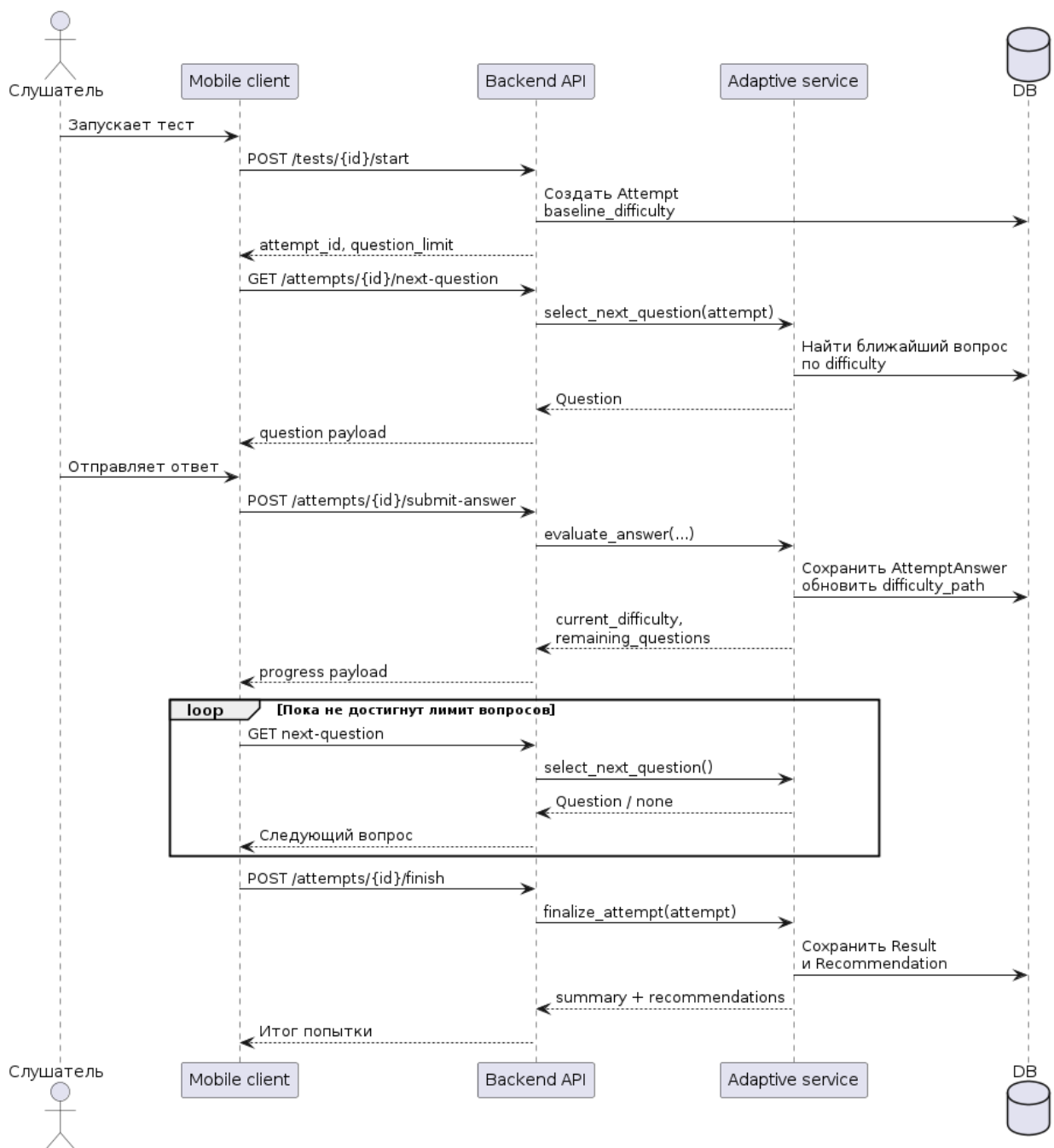


Рисунок 13 – Последовательность прохождения адаптивного теста

Для иллюстрации логики изменения сложности на рисунке 14 приведена схема алгоритма. Она отражает старт с базовой сложности, выбор ближайшего незаданного вопроса, изменение уровня сложности на шаг '+1/-1' и формирование рекомендаций после завершения попытки.

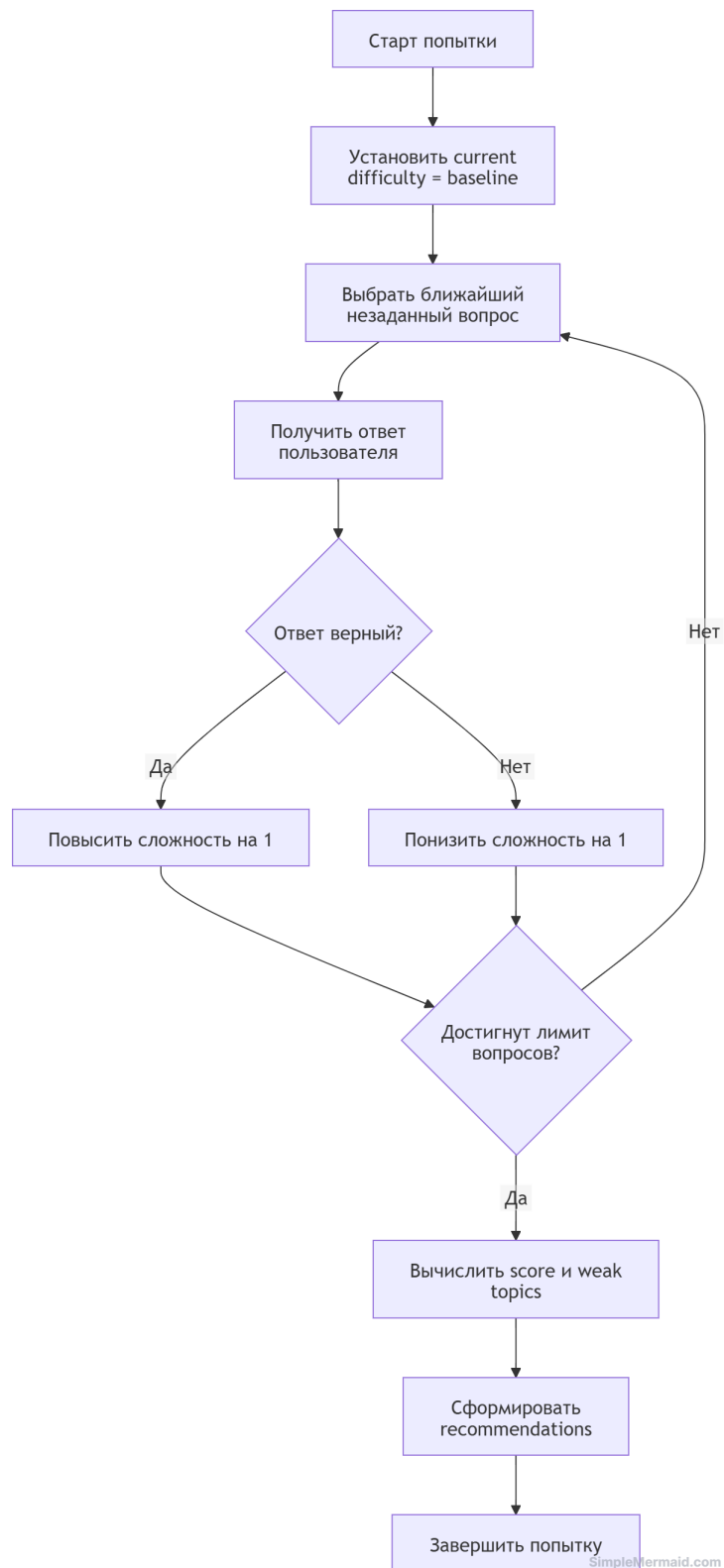


Рисунок 14 – Схема изменения сложности в адаптивном тестировании

2.6.3 Изоляция данных арендаторов

Multi-tenant изоляция данных реализована на трёх уровнях:

- а) на уровне запроса определяется код текущей организации;

б) на уровне авторизации проверяется, что пользователь имеет активное участие в этой организации;

в) на уровне доступа к данным выполняется фильтрация tenant-зависимых сущностей по полю `tenant_id`.

Таким образом, логическая изоляция не требует выделения отдельной базы данных на каждого арендатора и при этом обеспечивает корректное разделение данных в рамках учебного MVP.

2.7 Развёртывание и использование

Локальное развёртывание проекта выполняется с помощью Docker Compose. В контейнеризированный контур входят:

- а) backend-сервис;
- б) web-сервис;
- в) СУБД PostgreSQL.

Мобильный клиент в контейнеризированный контур не включён и запускается отдельно в среде Flutter, взаимодействуя с backend по HTTP API. Такой вариант развёртывания соответствует составу проекта и удобен для демонстрации на защите.

На рисунке 15 представлена схема локального развёртывания. Диаграмма отражает состав контейнеризированного контура и взаимодействие мобильного клиента с backend API вне Docker Compose.

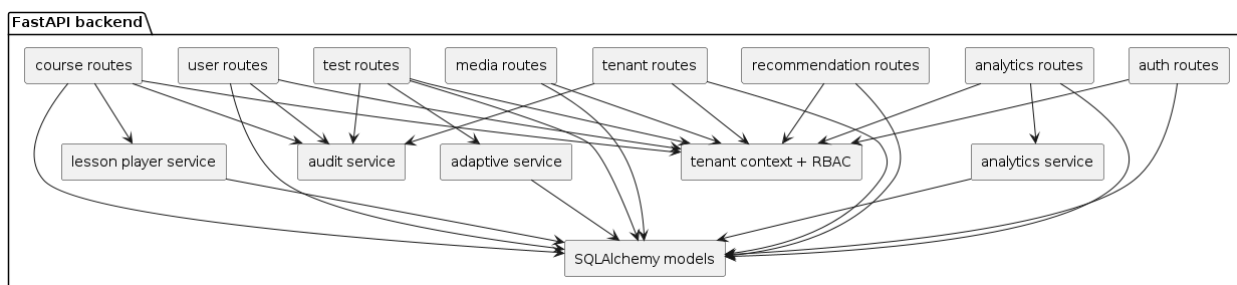


Рисунок 15 – Схема локального развёртывания системы

Для практического использования системы можно выделить две краткие инструкции:

а) администратор или преподаватель запускает стек, входит в веб-панель, создаёт пользователей, курсы, уроки и тесты, затем назначает курс слушателю;

б) слушатель запускает мобильное приложение, входит в систему, открывает назначенный курс, проходит урок и тест, после чего получает рекомендации.

Такой сценарий полностью соответствует критическому пути, который может быть положен в основу трёхминутного демонстрационного ролика на защите.

2.8 Выводы по главе

Во второй главе описана техническая реализация корпоративной LMS. Рассмотрены архитектура системы, формальные сценарии использования, инфологическая и даталогическая модели базы данных, проектирование интерфейсов, а также структурные схемы клиентской и серверной частей. Раскрыты состав REST API, логика адаптивного тестирования, механизм логической изоляции данных арендаторов и схема развёртывания программного комплекса. Представленные решения соответствуют фактической реализации проекта и обеспечивают основу для демонстрации работоспособности системы и её защиты.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Moodle lms. <https://moodle.org/>, 2026. [Электронный ресурс]. Дата обращения: 15.02.2026.
2. Vocal Media. Mobile learning solutions in 2024: Trends and innovations. <https://vocal.media/education/mobile-learning-solutions-in-2024-trends-and-innovations>, 2024. [Электронный ресурс]. Дата обращения: 15.02.2026.
3. Michael Feldstein et al. Adaptive learning systems: Surviving the storm. *EDUCAUSE Review*, 2016. [Электронный ресурс]. Дата обращения: 15.02.2026.
4. Yojji. Multi-tenant lms: The complete technical and business guide. <https://yojji.io/blog/multi-tenant-lms>, 2025. [Электронный ресурс]. Дата обращения: 15.02.2026.
5. Методические рекомендации по подготовке выпускных квалификационных работ (09.03.01, 09.03.03), 2021. Московский политехнический университет.
6. Opigno LMS. Finding the perfect mobile lms app: Features checklist. <https://www.opigno.org/blog/finding-perfect-mobile-lms-app-features-checklist>, 2024. [Электронный ресурс]. Дата обращения: 15.02.2026.
7. Moodle US. A guide to lms multi-tenancy: Do you need it? <https://moodle.com/us/news/lms-multi-tenancy/>, 2024. [Электронный ресурс]. Дата обращения: 15.02.2026.
8. Open lms: The open source lms that fits your needs. <https://www.openlms.net/>, 2026. [Электронный ресурс]. Дата обращения: 15.02.2026.
9. Moodle app. <https://apps.apple.com/app/moodle/id633359593>, 2026. [Электронный ресурс]. Дата обращения: 15.02.2026.
10. Святослав Александрович Куликов. *Тестирование программного обеспечения. Базовый курс*. 3 edition, 2024. Версия 3.2.6 от 24.05.2024.
11. Moodlecloud: Cloud lms. <https://moodlecloud.com/>, 2026. [Электронный ресурс]. Дата обращения: 15.02.2026.

12. Docebo Learning Network. Adaptive learning: What it is, benefits & how it works. <https://www.docebo.com/learning-network/blog/adaptive-learning/>, 2026. [Электронный ресурс]. Дата обращения: 15.02.2026.
13. Fastapi documentation. <https://fastapi.tiangolo.com/>, 2026. [Электронный ресурс]. Дата обращения: 15.02.2026.
14. Django rest framework. <https://www.django-rest-framework.org/>, 2026. [Электронный ресурс]. Дата обращения: 15.02.2026.
15. React: Documentation. <https://react.dev/>, 2026. [Электронный ресурс]. Дата обращения: 15.02.2026.
16. Typescript documentation. <https://www.typescriptlang.org/docs/>, 2026. [Электронный ресурс]. Дата обращения: 10.04.2026.
17. Flutter documentation. <https://docs.flutter.dev/>, 2026. [Электронный ресурс]. Дата обращения: 15.02.2026.
18. Sqlite documentation. <https://sqlite.org/docs.html>, 2026. [Электронный ресурс]. Дата обращения: 10.04.2026.
19. Postgresql documentation. <https://www.postgresql.org/docs/>, 2026. [Электронный ресурс]. Дата обращения: 15.02.2026.
20. Docker compose documentation. <https://docs.docker.com/reference/compose-file/>, 2026. [Электронный ресурс]. Дата обращения: 10.04.2026.